Day 1 = 1/2 Half day - 12 pm CT
1 Coffee break = 10:00 am CT — 15 mins

Day 2 & 3
2 Coffee breaks < 10:00 am CT, 2:30 pm CT > 15 mins
1 lunch break = [A. 11:30 am CT, B. 12:00 pm CT] > 1 Hr.

Github Copilot → "Generative AI Application"
↳ AI Coding Assistant

AI = Produce something New
New Content - Text, Code, Image, Audio, Video

"Generative AI" = Powered by (LLM) → Models
(Large Language Model)
(Subset of FM) ⟷ (Foundation Model)

FM → Pretrained Model + Trained on Internet scale of data

[Anthropic → Claude Opus, (Claude Sonnet), Claude Haiku
Meta → Llama
Google → Gemini
AI 21 → Jamba
OpenAI → GPT =

Model

Q → INPUT → Model → Output → output
# of tokens

Input embeddings ⇒ Prompt (Instruction)
Input → [Translate | This sentence into French]
[I | love | cats]
Tokens → (Numerical representation)
vectors → Model → output (Logits) # of tokens

Model < input, output > # of Tokens / Request

Claude Models (Chatty Models)

Q → Input → Github Copilot → Prompt → Model → Output
Prompt
Context
Output
Not Extensive Knowledge

① Inline Suggestions
② Command palette
③ Inline chat → Ctrl + I
+
Cmd + I
④ Copilot Chat — Agent Mode, Ask Mode
⑤ /slash commands ⇒ Shortcuts in the chat interface
↳ To perform specific common coding tasks.

Prompt Engineering for Copilot
✓ What changes vs Normal Prompting
✓ Best Practices + Prompt Patterns
✓ Debug-Style Prompts
✓ Common Mistakes + fixes
✓ Hands-on Lab.

* Copilot = "Jr. Engineer" ⇒ Need Context, constraints

Rule #
① TS / JS
② DON'T ACCEPT CODE BLINDLY. = Draft.

Copilot = Context + Constraints + TARGET output
① Context = Existing code, file structure, interfaces, examples
② Constraints = language, style, lib, performance, security
③ Target output = Function, Tests, Refactor, explanation, debugging steps.

PROMPT = Build an API ✗

Golden Prompt Templates
① Goal = What you want
② Input/output = function signature, data shape
③ Rules = Must / must not
④ Edge cases = nulls, empty arrays, invalid input
⑤ Examples = one / two
⑥ Deliverables = code + test + explanation

Best Practices
① Give Copilot — "The world it lives in"
- File it's edit
- nearby types / interfaces
- sample data
- existing pattern / repo
② Ask for incremental output, not a whole system
③ Force Transparency.
④ [Control Style & safety]

* Debug-Style Prompting = Framework
① Observe = error message + when it happens + min code.
② Hypothesize = Ask for 3 likely causes
③ Prove = Ask for checks / logs / tests to confirm
④ Fix = Apply min. change
⑤ Prevent

* Common Mistakes
① Vague Prompts
↳ Make it Better ✗
② Asking too much at a once
↳ API + DB + auth + UI ⇒ one go.
③ Not pinning dependencies / env.
④ Copying Code without verification